

INFRAESTRUTURA PARA BIG DATA

Virtualização, Containers e Big Data

Das máquinas virtuais à orquestração com Kubernetes: as tecnologias que sustentam o processamento de dados moderno

Prof. Claudio Nunes · Ciência de Dados



Roteiro da aula

1

Fundamentos da virtualização

Hipervisores, VMs e isolamento

2

Containers por dentro

O que isola um container e como a imagem é montada

3

VMs × Containers: como escolher

Cenários, comparativo e arquiteturas híbridas

4

Orquestração

Kubernetes e o modelo de estado desejado

5

Containers e Big Data

Spark, Kafka e pipelines em Kubernetes

6

Fechamento

Checklist de decisão, síntese e referências

Por que isso importa para Big Data?



"Na minha máquina funciona"

Versões diferentes de Python, Java, Spark e bibliotecas geram resultados inconsistentes entre desenvolvimento e produção.



Clusters, não computadores isolados

Big Data exige dezenas ou centenas de computadores (os nós) trabalhando juntos como um só sistema — é isso que chamamos de cluster. Configurar cada nó manualmente não escala.



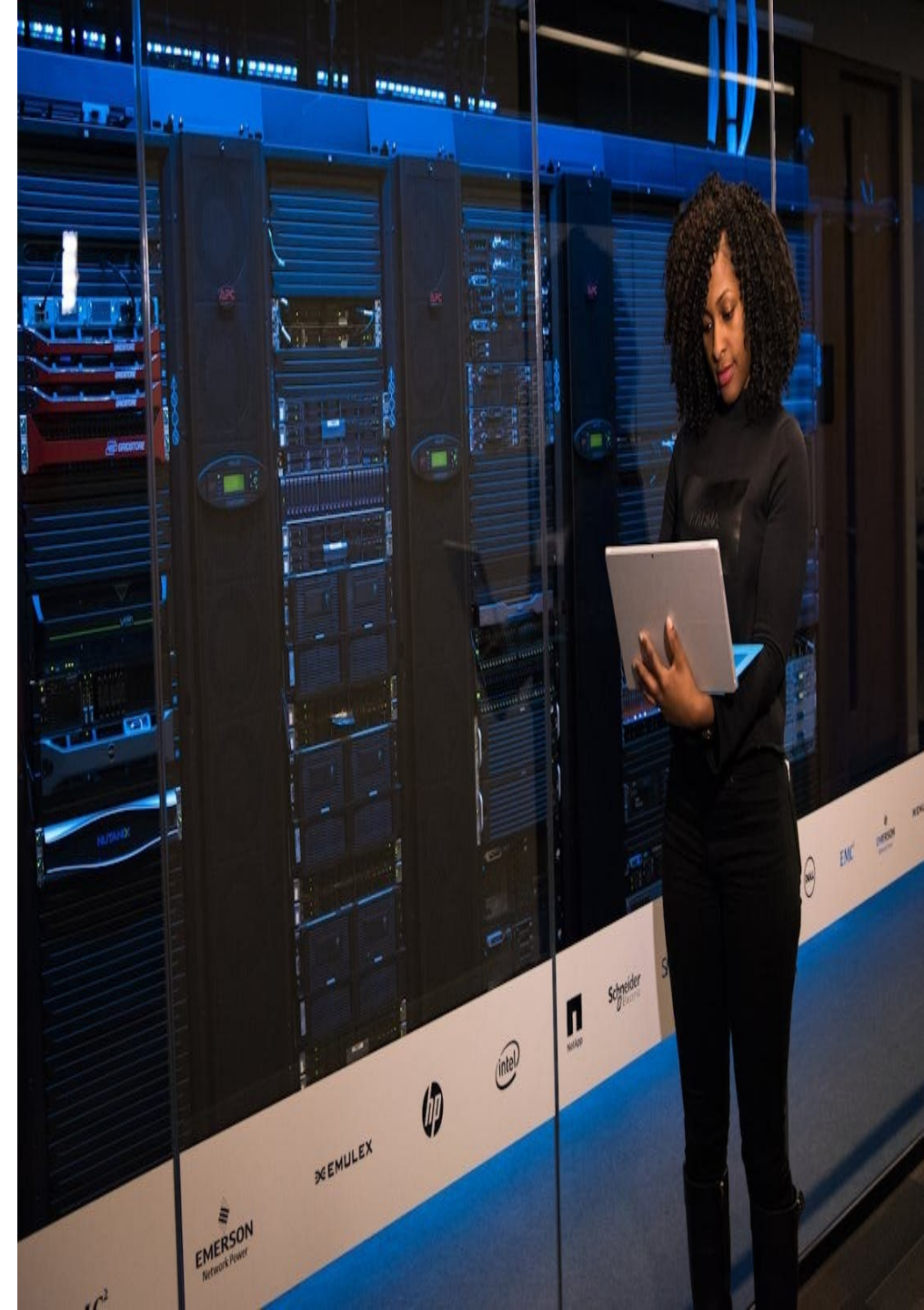
Carga que varia muito

Um processo noturno de ETL (extrair, transformar e carregar dados) pode precisar de 50 nós por 2 horas e de nenhum no resto do dia.



Custo e compartilhamento

Várias equipes disputam o mesmo hardware: é preciso isolar e medir o consumo de cada uma.



BLOCO 1

Fundamentos da virtualização

Como um único servidor físico se transforma em muitos computadores independentes



Duas formas de isolar cargas de trabalho



Virtualização (VMs)

Cria várias máquinas virtuais isoladas sobre o mesmo hardware físico.

Cada VM tem seu próprio sistema operacional completo — inclusive o kernel, o núcleo que controla CPU, memória, disco e rede — e se comporta como um computador independente.

Quem faz isso: o hipervisor, apresentado no próximo slide.



Containers

Isolam uma aplicação e suas dependências em um processo leve, que roda direto sobre o sistema operacional do host — a máquina física e o sistema que a controla.

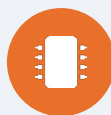
Todos os containers compartilham o mesmo kernel do host, mas cada um enxerga um ambiente separado.

Quem faz isso: namespaces e cgroups, dois recursos do próprio kernel Linux, detalhados no bloco 2.

Analogia: VMs são casas separadas, cada uma com sua fundação. Containers são apartamentos no mesmo prédio — fundação (kernel) compartilhada, portas trancadas.

O hipervisor: tipo 1 e tipo 2

Hipervisor: camada de software que reparte o hardware de um servidor e executa, ao mesmo tempo, vários sistemas operacionais convidados. Cada convidado se comporta como se tivesse a máquina só para si.



Tipo 1 • bare metal

Executa direto sobre o hardware, sem SO intermediário: o menor overhead possível — o consumo extra de CPU, memória e tempo que uma camada de software impõe.

Exemplos: VMware ESXi, Microsoft Hyper-V, KVM, Xen.

Uso: data centers e nuvens públicas.

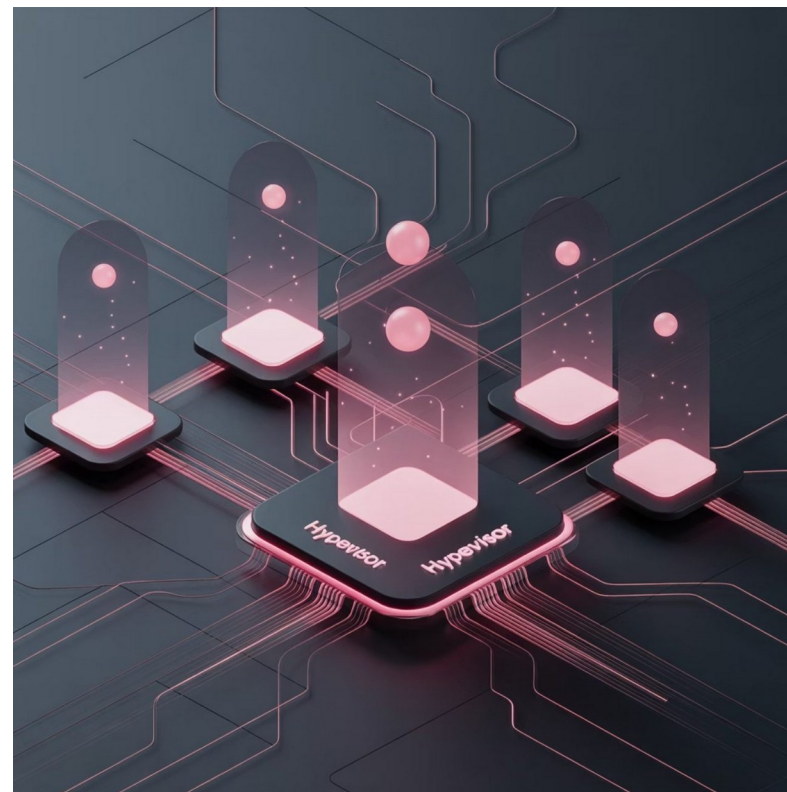


Tipo 2 • hosted

Executa como aplicativo sobre um SO hospedeiro (Windows, macOS, Linux).

Exemplos: VirtualBox, VMware Workstation, Parallels.

Uso: laboratório, estudo e testes no computador pessoal.



Na nuvem: ao criar uma instância (um servidor virtual alugado, como EC2, Azure VM ou Compute Engine), você recebe uma VM que roda sobre um hipervisor tipo 1.

Características das máquinas virtuais



Isolamento forte

Cada VM tem o seu próprio kernel. Uma falha ou invasão fica contida dentro daquela VM e não atinge as demais.



Recursos dedicados

CPU, memória e disco são reservados por VM — mais previsível, porém com maior consumo.



Flexibilidade de SO

Linux e Windows, em versões diferentes, lado a lado no mesmo servidor físico.



Inicialização mais lenta

Cada VM precisa carregar um sistema operacional inteiro ao iniciar — o boot: minutos, não segundos.



Compatibilidade ampla

Praticamente qualquer aplicação roda sem modificação, inclusive programas antigos e drivers de hardware específicos.



BLOCO 2

Containers por dentro

O que realmente acontece quando você executa docker run



Conceito de containers

Um container é um processo comum do Linux, empacotado junto com suas dependências e isolado por mecanismos que já existem dentro do kernel.



Imagem

Pacote pronto e imutável, organizado em camadas de arquivos. Segue o padrão aberto OCI (Open Container Initiative), por isso funciona em qualquer runtime.



Runtime

Programa que transforma a imagem em processo em execução: containerd ou CRI-O, apoiados no runc. O Docker usa o containerd por baixo.



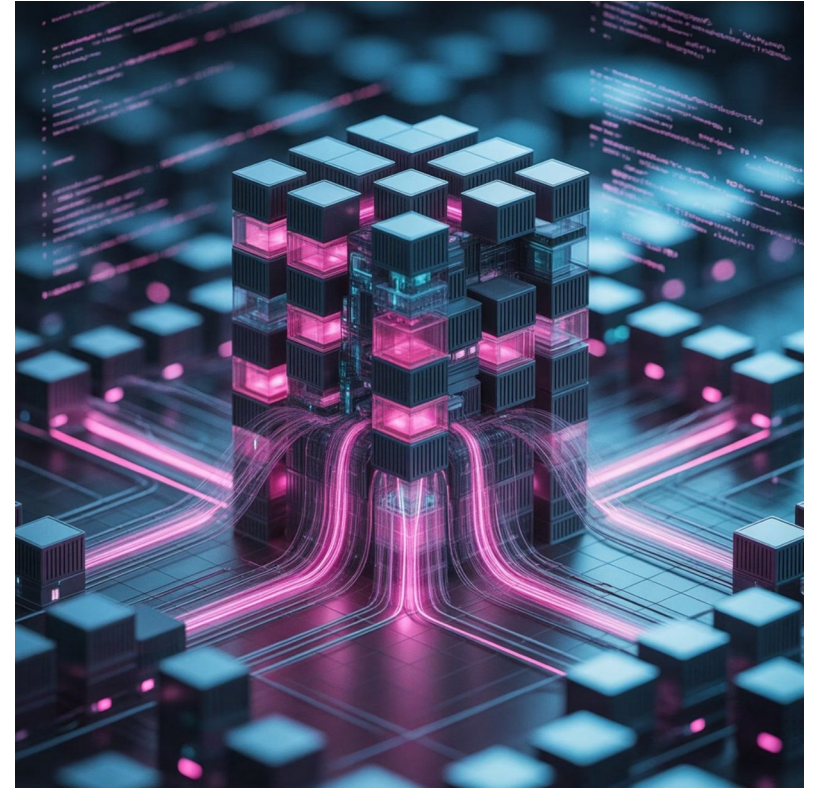
Registry

Servidor onde as imagens ficam guardadas, de onde são baixadas (pull) e para onde são enviadas (push): Docker Hub, GitHub Container Registry, ECR, ACR.



Orquestrador

Software que gerencia muitos containers em muitos nós ao mesmo tempo: Kubernetes, Nomad, k3s (laboratório). É o tema do bloco 4.



Os dois pilares: namespaces e cgroups

Namespaces — o que o processo consegue VER

Namespace	Isola
pid	árvore de processos própria (o app é o PID 1)
net	interfaces, IPs e portas próprios
mnt	sistema de arquivos próprio
uts	hostname próprio
ipc	memória compartilhada isolada
user	mapeia root do container para usuário comum no host

cgroups — o que o processo consegue USAR



CPU

Limita quantos núcleos — ou frações de núcleo — o container pode usar.



Memória

Define um teto de memória. Se o processo passar do teto, o kernel o encerra (erro OOMKilled).



I/O de disco

Controla vazão de leitura e escrita.

```
docker run --cpus=2 --memory=4g \  
jupyter/pyspark-notebook
```

Atenção: namespaces e cgroups apenas repartem o kernel do host — nenhum deles cria um kernel novo para o container.

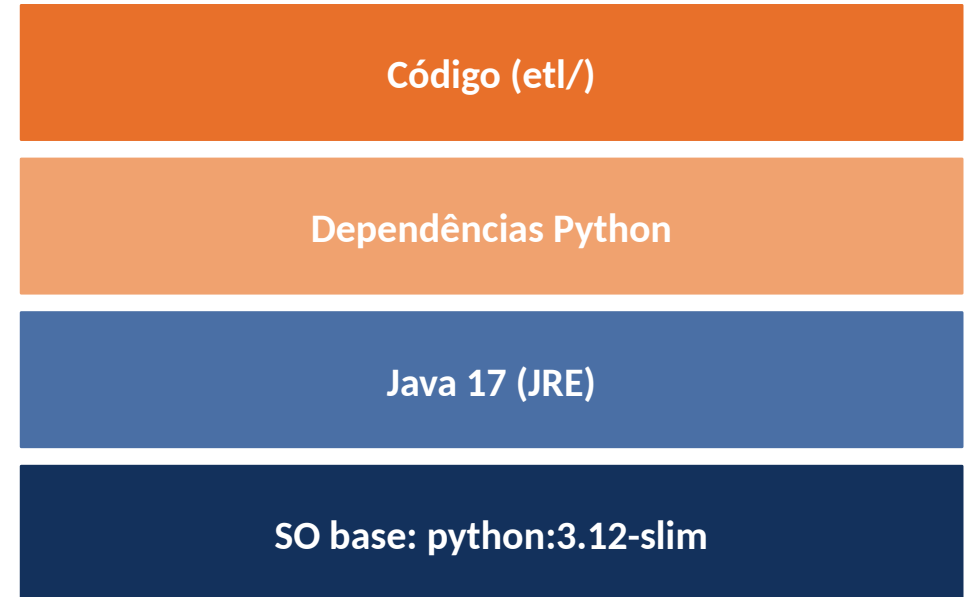
Imagens, camadas e o padrão OCI

```
# Dockerfile de uma tarefa PySpark
FROM python:3.12-slim          # camada 1: SO base
RUN apt-get update && \
    apt-get install -y openjdk-17-jre # camada 2
COPY requirements.txt .
RUN pip install -r requirements.txt # camada 3
COPY etl/ /app/etl/              # camada 4: código
CMD ["python", "/app/etl/main.py"]
```



Imutável e versionada

A versão marcada no nome (etl:1.4.2) garante que a produção executa exatamente a imagem testada. Tudo o que está nas camadas viaja com ela e pode ser lido por quem a baixa.



As camadas da imagem são somente leitura. Ao iniciar, o container recebe por cima uma camada gravável — e ela é apagada junto com o container (docker rm).

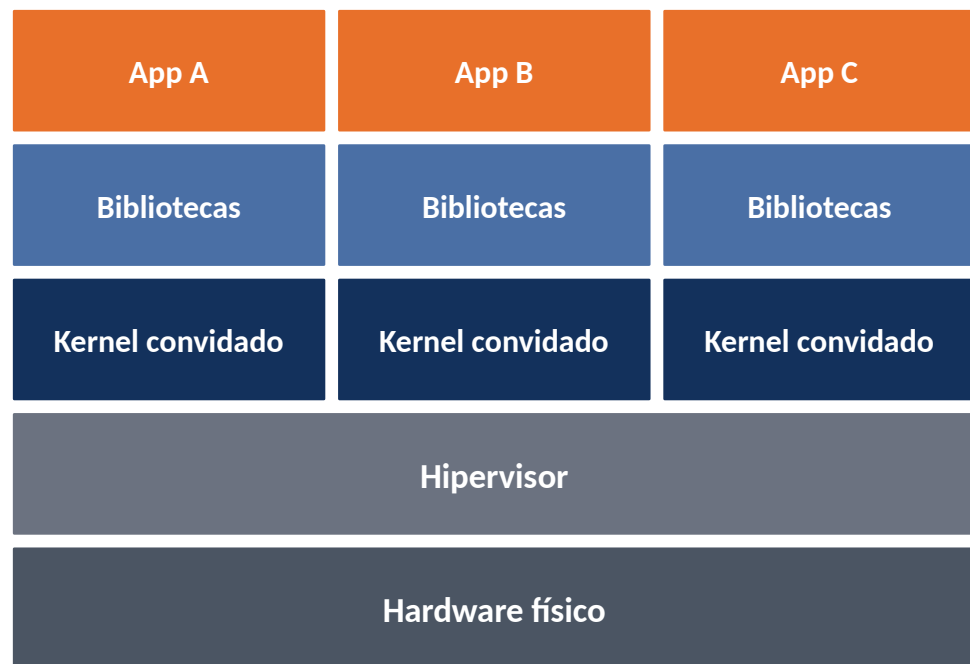


Cache de build

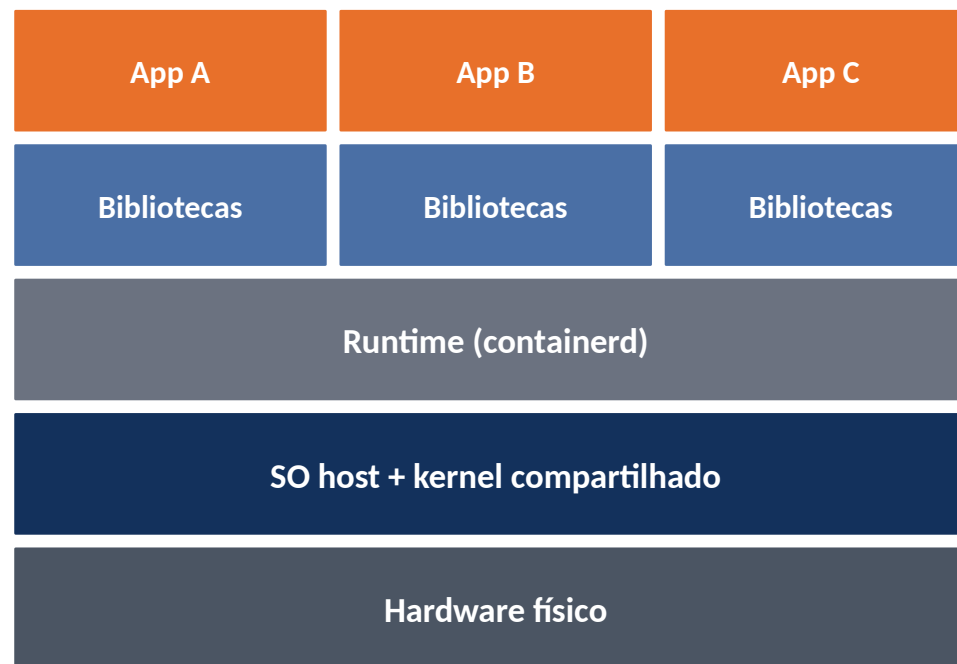
Ao reconstruir, só as camadas alteradas são refeitas: coloque o que muda mais, o código, por último.

Arquitetura em camadas: VM × Container

Máquinas virtuais



Containers



Sem kernel convidado:
menos overhead,
início em segundos

mas depende do
kernel do host

Implicação: sem kernel convidado, o processo do container entrega suas chamadas de sistema — os pedidos que todo programa faz ao kernel para abrir arquivos, usar a rede, criar processos — direto ao kernel do host. Ganha velocidade e densidade, mas só roda se for compatível com esse kernel.

Características dos containers



Leveza e eficiência

Kernel compartilhado: megabytes em vez de gigabytes, e muito mais instâncias por host.



Portabilidade

A imagem leva tudo de que a aplicação precisa: roda igual no notebook, no servidor de testes e na nuvem.



Implantação rápida

Inicia e para em segundos, o que viabiliza escalar sob demanda.



Efêmeros por natureza

A camada gravável some junto com o container. O que precisa durar vai para um volume — área de armazenamento gerenciada fora do container (`docker run -v dados:/caminho`) — ou para um armazenamento externo.



Isolamento mais fraco

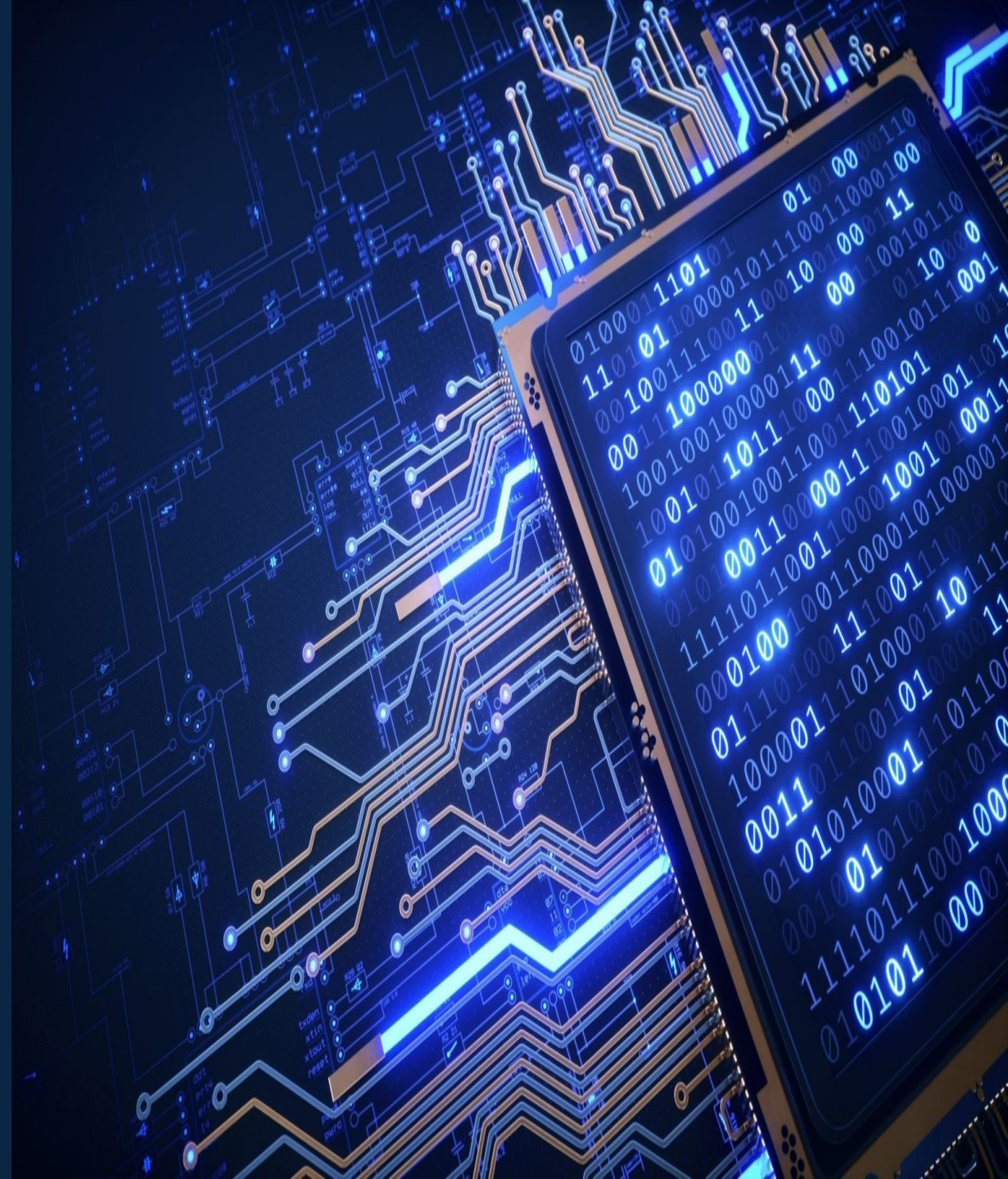
Como o kernel é único, uma falha nele afeta todos os containers do host: exige configuração cuidadosa de segurança.



BLOCO 3

VMs × Containers: como escolher

Não é uma disputa: cada tecnologia resolve um problema diferente — e, na prática, elas se combinam



Cenários típicos de uso

Virtualização



Ambientes multissistema

Testes de software em vários SOs e simulação de redes.



Consolidação de servidores

Muitas cargas em poucos hosts físicos, com menos energia e espaço.



Aplicações críticas isoladas

Fronteira forte para sistemas sensíveis ou de terceiros.



Infraestrutura de nuvem (IaaS)

IaaS = infraestrutura como serviço: alugar máquinas virtuais prontas (EC2, Azure VM, Compute Engine).

Containers



Desenvolvimento e testes

Ambiente idêntico do notebook à produção, em testes e entregas automatizados (CI/CD) sempre reproduzíveis.



Microserviços

A aplicação é dividida em serviços pequenos: cada um em seu container, escalado e atualizado à parte.



Experimentação em dados

Subir Spark, Kafka ou um notebook em minutos e descartar depois.



Jobs em lote e pipelines

Job é uma tarefa com começo e fim, como um ETL ou o treinamento de um modelo; pipeline é a sequência dessas tarefas. Nascem, executam e terminam.

Comparativo geral

Aspecto	VMs	Containers
Isolamento	Forte (kernel dedicado)	Médio-forte (namespaces/cgroups)
Provisionamento	Minutos (boot do SO)	Segundos (início de processo)
Densidade por host	Dezenas	Centenas
Tamanho típico	Gigabytes	Megabytes
Portabilidade	Entre hipervisores/nuvens	Altíssima pela imagem — desde que o kernel do destino seja do mesmo tipo
Compatibilidade	Qualquer SO, app ou driver	Apps compatíveis com o kernel do host
Gestão	Imagens-modelo de VM, mantidas manualmente	Imagens versionadas em registry, publicadas por automação (CI/CD)
Monitoramento	Acompanha cada VM e cada host	Acompanha cada container e cada serviço
Segurança	Fronteira forte por VM	Depende do kernel do host e de como ele é configurado

Os números são ordens de grandeza, não limites: dependem do hardware e da carga.

Quando escolher VMs



Isolamento e compliance

Regulamentações rigorosas, auditoria detalhada e dados muito sensíveis, que exigem fronteiras de segurança bem definidas — por exemplo, dados pessoais sensíveis protegidos pela LGPD (Lei Geral de Proteção de Dados).



Drivers e kernel específicos

Placas de rede especiais, GPUs com drivers próprios, ajustes profundos no kernel ou a necessidade de um sistema operacional diferente do host.



Sistemas legados

Programas antigos e monolíticos, difíceis de empacotar em uma imagem sem reescrever boa parte do código.



Hadoop com HDFS

Clusters Hadoop que guardam os dados no HDFS — sistema de arquivos distribuído sobre os discos dos próprios nós — e processam ali mesmo: precisam de máquinas fixas e desempenho previsível.

Quando escolher containers



Elasticidade e agilidade

Tarefas em lote, ETL e treinamento de modelos: pedem recursos depressa, crescem sob demanda e devolvem tudo ao terminar, o que reduz custo.



Reprodutibilidade

Imagens imutáveis versionam pipelines com precisão e garantem ambientes idênticos em dev, teste e produção.



Vários times no mesmo cluster

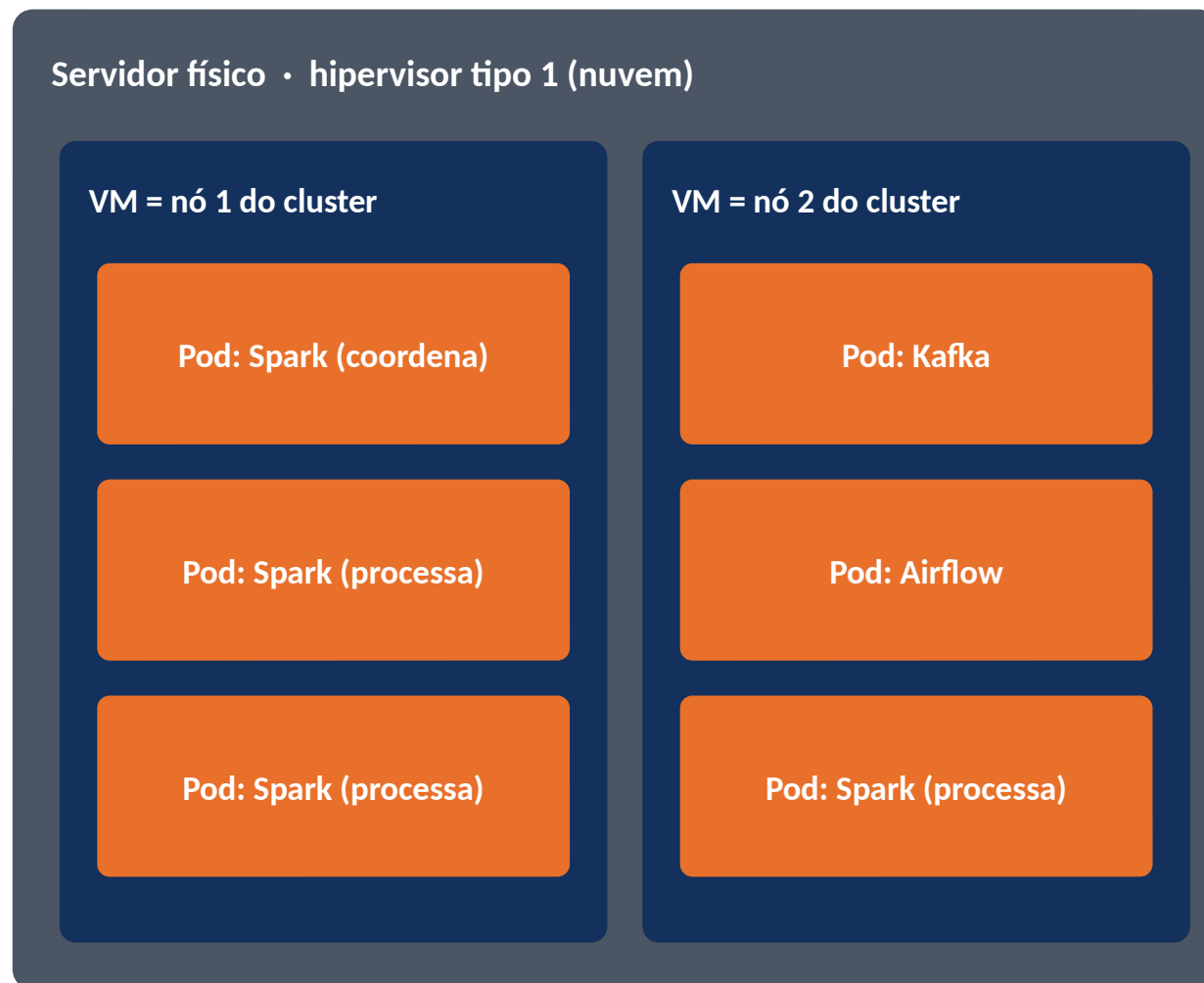
Notebooks e serviços de dados de equipes diferentes convivem no mesmo cluster, separados por namespaces do Kubernetes — divisões lógicas do cluster, que não se confundem com os namespaces do Linux do bloco 2 — e por cotas de recurso.



Automação por operadores

Operador é um programa que roda no cluster e cuida sozinho de instalar, escalar, atualizar e recuperar um sistema complexo: Spark Operator, Strimzi (Kafka), Flink Operator.

Na prática: as duas coisas juntas



Kubernetes gerenciado

EKS, AKS e GKE rodam os containers dentro de VMs: a VM isola os clientes, o container dá a agilidade. Pod é a menor unidade do Kubernetes — um ou mais containers que rodam juntos (bloco 4).



MicroVMs

VMs mínimas que iniciam em frações de segundo e unem a praticidade do container ao isolamento da VM: Firecracker (AWS Lambda) e Kata Containers.



No seu computador

Docker Desktop e WSL2 executam uma VM Linux leve para hospedar os containers.

BLOCO 4

Orquestração de containers

De um container no notebook a milhares em produção



O que é orquestração?

Gerenciamento automático de muitos containers, distribuídos em muitos nós, para entregar uma aplicação. O orquestrador age sobre os containers, nunca sobre as máquinas: não liga nem desliga hardware.

Modelo declarativo (estado desejado)

Você descreve o que quer — por exemplo, 3 réplicas (3 cópias idênticas) desta imagem, com 2 CPUs cada — e o orquestrador trabalha sem parar para que a realidade fique igual à declaração. Esse ciclo é o loop de reconciliação.



Escalonamento (scheduling)

Decide em qual nó cada container vai rodar, conforme os recursos livres em cada um.



Rede e descoberta de serviços

Containers se encontram por nome, não por IP.

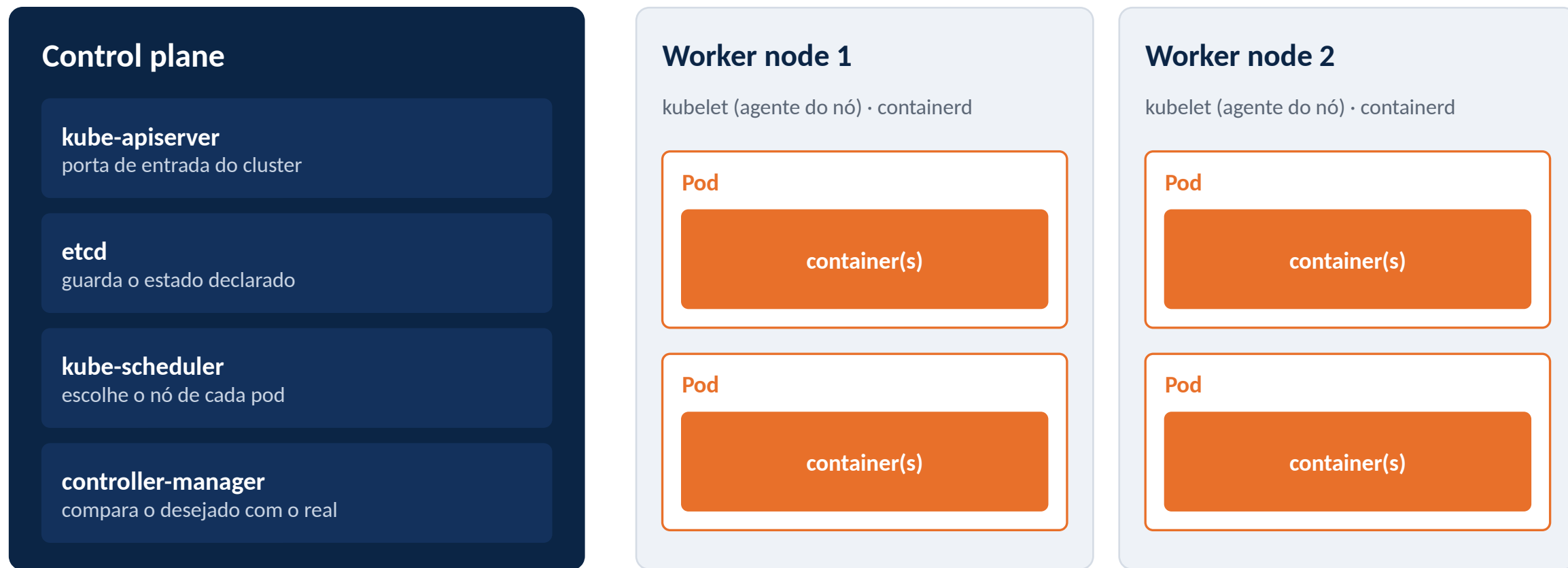


Ciclo de vida

Implantar, atualizar, aumentar ou reduzir a quantidade e remover — tudo automatizado.



Kubernetes: arquitetura essencial



Pod menor unidade: 1+ containers

Deployment cópias iguais, sem dados próprios

StatefulSet cópias com identidade e disco próprios

Job / CronJob tarefa que roda até terminar

Service endereço estável + balanceamento

PVC pedido de disco (PersistentVolumeClaim)

Ferramentas de orquestração



Kubernetes (K8s)

Padrão de mercado, mantido pela CNCF — a fundação que abriga os principais projetos de software para nuvem. Versões gerenciadas pelos provedores: EKS, AKS, GKE. Versões leves para laboratório: k3s, kind, minikube.



Docker Swarm

Orquestrador que já vem com o Docker. Trata vários hosts como se fossem um só e é simples de aprender. Boa porta de entrada, mas com ecossistema bem menor.



Red Hat OpenShift

Distribuição corporativa de Kubernetes: adiciona CI/CD integrado, console para desenvolvedores, políticas de segurança mais rígidas e suporte comercial.



HashiCorp Nomad • Apache Mesos

Nomad: orquestrador simples, que distribui containers, VMs e programas comuns. Mesos + Marathon: pioneiros em clusters de Big Data, hoje em forte declínio — tome-os apenas como referência histórica.

O que a orquestração entrega



Escalabilidade automática

Aumenta e reduz o número de réplicas conforme o uso de CPU, de memória ou o tamanho de uma fila de mensagens. Outro componente acrescenta e remove nós do cluster.



Balanceamento de carga

O Service distribui as requisições entre as réplicas que estão saudáveis, o que melhora a disponibilidade.



Implantação contínua

A versão nova entra aos poucos, substituindo as réplicas antigas sem tirar o serviço do ar — e é possível voltar à versão anterior se algo der errado.



Recuperação automática

O cluster verifica periodicamente se cada container responde; quando algum falha, ele é reiniciado ou recriado em outro nó.

BLOCO 5

Containers e Big Data

Como a orquestração muda a forma de executar Spark, Kafka e pipelines de dados



Benefícios para projetos de Big Data



Escalabilidade elástica

Recursos crescem e encolhem com o volume de dados, sem pagar por capacidade ociosa.



Consistência e portabilidade

Mesmo ambiente em desenvolvimento e em produção; o mesmo pipeline roda na nuvem ou em servidores próprios.



Agilidade nas entregas

Modelos e pipelines atualizados continuamente, por automação, sem interromper o serviço.



Experimentação

Testar um novo framework custa minutos; o ambiente é descartável.



Uso eficiente de recursos

Alocação dinâmica e alta densidade reduzem custo de processamento intensivo.



Vários times, um cluster

Divisões lógicas e cotas de recurso evitam que uma equipe consuma o cluster inteiro e prejudique as outras.

O ecossistema de dados no Kubernetes

Orquestração de pipelines

Apache Airflow · Argo Workflows · Dagster — agendam e encadeiam as etapas de um pipeline

Processamento

Apache Spark (suporte nativo desde a versão 3.1) · Spark Operator · Flink Kubernetes Operator

Fluxo contínuo de dados

Apache Kafka, instalado e operado pelo Strimzi

Consulta e análise

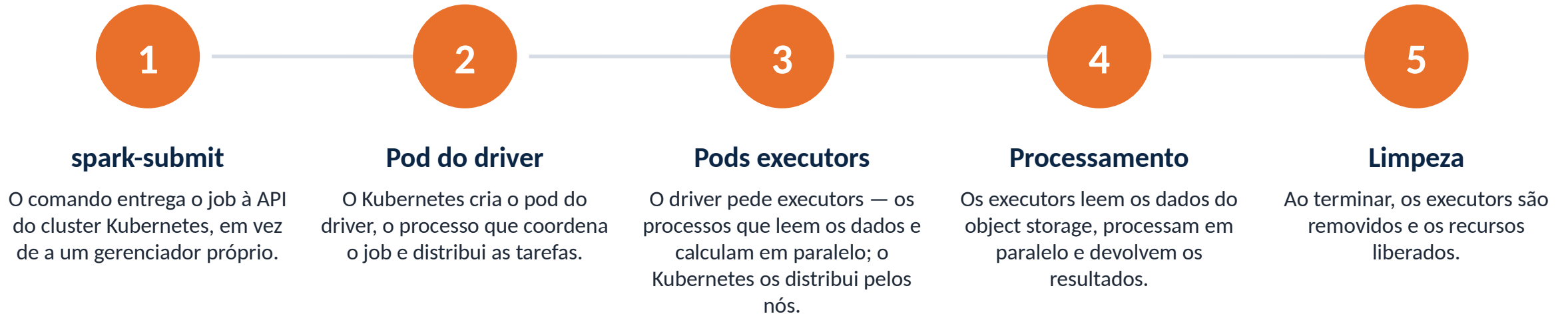
Trino (consultas SQL) · JupyterHub (notebooks) · MLflow (registro de experimentos)

Armazenamento

Object storage — arquivos guardados fora dos nós e buscados pela rede por uma chave (S3, MinIO, ADLS) · PersistentVolumes para disco no cluster · formatos de tabela (Iceberg, Delta)

Kubernetes · gerenciado pelo provedor (EKS, AKS, GKE) ou instalado em servidores próprios

Como o Spark roda no Kubernetes



```
spark-submit \  
  --master k8s://https://<api-server>:6443 \  
  --deploy-mode cluster \  
  --conf spark.kubernetes.container.image=registry/etl:1.4.2 \  
  --conf spark.executor.instances=10 \  
  --conf spark.executor.memory=4g \  
  --conf spark.dynamicAllocation.enabled=true \  
  local:///app/etl/main.py
```



Alocação dinâmica

O Spark cria e remove executors conforme o tamanho da fila de tarefas.



Spark Operator

Descreva o job em um arquivo de configuração e o operador cuida de todo o resto.

Desafios e cuidados



Estado e persistência

Bancos de dados e o Kafka precisam de identidade fixa e disco próprio (StatefulSet + PersistentVolume). Operadores maduros reduzem o risco.



Rede e shuffle

O shuffle do Spark — a troca de dados entre executors — gera muito tráfego; a rede virtual acrescenta atraso.



Perda de data locality

Com os dados em object storage, o processamento deixa de acontecer no mesmo nó onde o dado está: compense com formatos colunares e cache.



Segurança da cadeia

Imagens de origem confiável, varredura de vulnerabilidades e container sem privilégios de administrador. Senhas ficam fora da imagem, em um Secret — objeto do Kubernetes que entrega o segredo ao container só na hora de executar.



LGPD

Separe dados pessoais em namespaces próprios, criptografe volumes e controle quem acessa o quê com RBAC — regras de acesso por papel, que definem inclusive quem pode ler um Secret. Cuidado com dados pessoais nos registros de log.



Complexidade operacional

Kubernetes tem curva de aprendizado alta: avalie serviços gerenciados antes de montar o seu.

BLOCO 6

Fechamento

Checklist de decisão, síntese e referências



Checklist de decisão

Se a resposta for SIM...	...tenda a
Precisa de outro SO ou kernel específico?	VM
Há exigência regulatória de isolamento forte?	VM (ou microVM)
A carga é curta, repetitiva ou varia muito?	Container
Reprodutibilidade entre ambientes é crítica?	Container
A aplicação é legada e difícil de empacotar?	VM
Há várias equipes compartilhando o cluster?	Container + divisões lógicas e cotas
Os dados precisam de disco local e previsível?	VM ou StatefulSet com disco local
A equipe tem maturidade para operar Kubernetes?	Se não: serviço gerenciado

O que levar desta aula

01

VM virtualiza hardware; container virtualiza a visão do sistema operacional.

02

Container é um processo isolado por namespaces e limitado por cgroups.

03

Imagens são imutáveis; containers são efêmeros — dados vão para volumes ou object storage.

04

Orquestração = estado desejado + loop de reconciliação.

05

Em Big Data, o padrão moderno é processamento efêmero e elástico sobre armazenamento durável e separado.

06

Na prática, VMs e containers convivem: a escolha é por camada, não por disputa.

Referências e próximos passos



KLEPPMANN, M. *Designing Data-Intensive Applications*. O'Reilly.
Capítulos sobre sistemas distribuídos e processamento em lote.



ÖZSU, M. T.; VALDURIEZ, P. *Principles of Distributed Database Systems*. Springer.
Fundamentos de distribuição e fragmentação de dados.



Documentação oficial: docs.docker.com • kubernetes.io/docs
Conceitos (Pods, Deployments, StatefulSets) e tutoriais interativos.



Apache Spark: Running on Kubernetes — spark.apache.org/docs/latest/running-on-kubernetes.html
Configurações, imagens e alocação dinâmica.



Open Container Initiative (opencontainers.org) e CNCF Landscape (landscape.cncf.io)
Padrões de imagem/runtime e panorama do ecossistema.